

DMP01 FAQs

1. What is the difference between parallel execution and serial execution?

We can run Python code in two ways:

- Serially, where each code statement executes one after another. This is the most common way of writing code and accomplishes almost any task.
- In parallel, where multiple code statements execute simultaneously. Code for parallel execution involves working with threading and coroutines in Python and is relatively complex compared to the serial execution. Note that thread execution in Python is pseudo-parallel because the Python engine or interpreter does not support real parallelism, what the GIL (Global Interpreter Lock) does is to share slices of processor time for each thread. It is possible to achieve real parallelism by changing the GIL or with dedicated libraries.

2. Is Python generally used for live trading or only for research/backtesting purposes? Do we need to recode our strategies to other languages once we are done with the research part?

Python can be and is used for both purposes: research and live trading. There are two parts to this question, both from the trading frequency perspective. Python is sufficient for LFT and MFT space (where most retail traders are involved) and does the work. Generally, LFT and MFT traders do not recode their strategies from Python to C++. Python does the job very well there. For HFT, C++ is the choice. Again, this is with reference to speed and all other parameters being constant.

3. From where can we get historical options data?

The vendors' list would change depending on the market and the geography. Most of the time, it can be obtained from the broker or authorised data vendors. Please see [this](#) for a list of various data vendors.

4. How can we get access to intraday data with Python?

Get data (intraday/interday) in a CSV file or any other format. You can load it in pandas directly using the file reading functions like the `.read_csv()` function. Otherwise, if you connect directly to a data vendor using its API, it would mostly provide the function that we can use to fetch data. For example, we use yfinance API and use its `download()` function to fetch historical data.

5. When do we use `cumprod()` and `cumsum()` in the strategy returns calculation?

We can calculate strategy returns using either log returns or percentage returns. We use

- `cumsum()` function when we work with log returns, and
- `cumprod()` function when we work with percentage returns.

6. Why is it required to shift the position by 1 for calculating the strategy returns?

Because if we take a position in a particular bar (we assume that we take the position at the close of the bar), the returns will only be acquired from the next bar onwards. Hence before multiplying with the returns, we shift the position down by 1 place.

7. Is there a rule-of-thumb on when we should create functions instead of directly coding?

There is no such rule. Whenever we have a code that can be reused, or we need to perform the same operation multiple times, then we should consider defining a function for that operation. It makes the code modular and non repetitive.

Functions must perform single and individual tasks and must be flexible through their input parameters. Generally you will have a collection of functions that perform atomic tasks.

8. How can we calculate adjusted prices?

We can first extract the adjustment factor by comparing the Close and adjusted close prices and use the same factor on the remaining columns. Refer to SFM-01 class for the exact calculations. Most often, we get adjusted prices directly from the vendor.

9. Are there specific situations where EMA performs better than SMA and vice versa?

Generally speaking, when researchers want the indicator to be more sensitive to recent price changes, they use EMA. However, such ideas need to be validated before using them by backtesting them.

10. How can we trade right at the close price?

That is a simplifying assumption we make for the sake of backtesting. In reality, we will take position, say, 1 min before the market closes.

11. Instead of exporting data to CSV files for multiple stocks, how can we handle it in pandas DataFrame?

We can use a multi-index dataframe to store data for multiple stocks, and then we can save it as a pickle file using the **pd.to_pickle()** function of pandas. The main idea behind exporting the data to CSV files or any other file format is to save the data on persistent data storage.